Creating Electronic Books with Code and Artificial Intelligence Assistants

Course for the Faculties of Sciences and Chemical Sciences

Álvaro Lozano Murciego and André Filipe Sales Mendes

Universidad de Salamanca

1st Edition • 2026 • DOI: 10.5281/zenodo.20449102

CONTENTS

Welcome to the **HTML Guide for Computer Engineering**.

This book will teach you the essential knowledge you need as a Computer Engineering student about **HTML (HyperText Markup Language)**. It's not an exhaustive manual, but a pragmatic introduction to the concepts and tools every computer engineer should master.

What is HTML?

HTML is the **standard markup language** for creating web pages. It provides the structure and meaning of web content, while CSS controls presentation and JavaScript adds interactivity.

As a computer engineer, understanding HTML is fundamental because:

- **It's the foundation of the web**: every web page, web application, and modern digital content is built on HTML.
- **It's simple yet powerful**: you can create structured content clearly and semantically.
- **It enables communication with other professionals**: designers, frontend developers, and accessibility specialists all speak the language of HTML.
- **It opens doors**: understanding HTML is the first step toward web development, progressive web apps (PWA), electronic documents, and beyond.

What will you learn?

In this book we will cover:

1. **Fundamental Concepts —** What tags are, attributes, and how an HTML document is structured.
2. **The HTML Document —** The anatomy of a page: DOCTYPE, head, body, meta tags, favicon, and SEO.
3. **Semantic Elements —** How to use the correct tags to give meaning to your content (headings, paragraphs, lists, citations).
4. **Links and Hyperlinks —** How to connect documents and resources with URLs, absolute and relative paths.
5. **Tables —** How to structure tabular data correctly.
6. **Your First HTML —** A practical project to bring it all together.

How to use this book

- Each chapter includes **practical examples** you can copy and experiment with.
- Concepts are ordered **from simple to complex**.
- It's recommended to follow the sequential order: each chapter builds on the previous ones.

Tools you'll need

- A **code editor** (VS Code, Sublime Text, Atom, etc.)
- A **web browser** (Chrome, Firefox, Safari, Edge)
- **Basic command line** (Git Bash, PowerShell, or Terminal)

That's all. HTML doesn't require complicated installations.

Note for educators

If you're a teacher using this material for your course, you can:

- Adapt it to your curriculum
- Add your own exercises
- Create PDF versions for printing
- Publish it on your own web server

Consult the TeachBook documentation in the resources section.

Let's get started!

HTML FUNDAMENTAL CONCEPTS

In this section you will learn the basic concepts you need to understand to work with HTML. It's not complicated, but it's important to grasp these ideas well from the beginning.

1.1 What is a tag?

A **tag** is the basic unit of HTML. It's code that tells the browser how to display or interpret content.

A typical tag has this structure:

```
<tagname>content</tagname>
```

Example:

```
<p>This is a paragraph.</p>
```

Here, `<p>` is the **opening tag**, `This is a paragraph.` is the **content**, and `</p>` is the **closing tag**. The browser interprets this as a paragraph.

1.2 Self-closing tags

Some tags have no content and close themselves. They're called **empty tags** or **self-closing tags**:

```
<img src="image.png" alt="Description">
<br>
<input type="text">
```

These tags don't need a separate closing tag.

1.3 What are attributes?

Attributes are additional properties that add information to a tag. They're placed inside the opening tag.

```
<img src="image.png" alt="Image description">
```

In this example:

- `src` is the attribute that specifies the **path** to the image

- `alt` is the attribute that provides **alternative text** if the image can't be loaded

Attributes always have the form `name="value"`.

1.4 Nesting

Tags can contain other tags. This is called **nesting**. Example:

```
<div>
  <h1>Main Title</h1>
  <p>This paragraph is inside the div.</p>
</div>
```

Here, `<h1>` and `<p>` are **nested** inside the `<div>`. Order matters: the tag that opens first must close last.

1.5 Basic structure of an HTML document

Every HTML page has a minimal structure:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Page title</title>
  </head>
  <body>
    <h1>Content here</h1>
  </body>
</html>
```

- `<!DOCTYPE html>` — tells the browser it's an HTML5 document
- `<html>` — the root tag that contains everything
- `<head>` — contains information about the page (not visible in the browser)
- `<body>` — contains the visible content of the page

1.6 Comments

Comments are annotations in the code that the browser ignores. They're useful for documenting your work:

```
<!-- This is a comment -->
<p>This text is visible</p>
<!-- You can explain here what this section does -->
```

Comments don't appear on the page, only in the source code.

1.7 Special characters

Sometimes you need to display special characters in HTML. Use **HTML entities**:

Character	Code	Use
<	<	Less than
>	>	Greater than
&	&	Ampersand
"	"	Double quotes
'	'	Single quotes
©	©	Copyright symbol
€	€	Euro symbol

Example:

```
<p>The price is &lt; 10€ and the formula is E &gt; mc²</p>
```

Following chapters will cover each of these concepts in depth.

1.8 Detailed Anatomy of Tags

Understanding the complete structure of a tag is fundamental for working correctly with HTML.

1.8.1 Parts of a tag

```
<p class="important" id="paragraph-1">This is content</p>
```

1. **Opening symbol** (<) — marks the start
2. **Tag name** (p) — defines what type of element it is
3. **Attributes** (class="important" id="paragraph-1") — additional information
4. **Closing symbol** (>) — ends the opening tag
5. **Content** (This is content) — what is displayed or processed
6. **Closing tag** (</p>) — marks the end of the element

1.8.2 Common tag types

Containers

Tags that contain other tags or content:

```
<div>...</div>     <!-- Generic block -->
<section>...</section>  <!-- Semantic section -->
<article>...</article>  <!-- Article -->
<nav>...</nav>     <!-- Navigation -->
<header>...</header>    <!-- Header -->
```

(continues on next page)

(continued from previous page)

```
<footer>...</footer>    <!-- Footer -->
<main>...</main>   <!-- Main content -->
```

Text tags

Tags that format text:

```
<h1>...</h1>  <!-- Heading level 1 -->
<p>...</p>    <!-- Paragraph -->
<strong>...</strong>  <!-- Strong text (important) -->
<em>...</em>  <!-- Emphasis -->
<small>...</small>    <!-- Small text -->
<span>...</span>  <!-- Generic inline text -->
```

Empty (self-closing)

They don't contain content or a closing tag:

```
<img src="..." alt="...">
<input type="...">
<br>
<hr>
<meta name="..." content="...">
<link rel="..." href="...">
```

1.8.3 Hierarchy and correct nesting

Tags must be nested correctly:

```
<!-- ✓ CORRECT: Elements close in order -->
<div>
  <p>
    <strong>Important text</strong> in a paragraph
  </p>
</div>

<!-- □ INCORRECT: Tag overlap -->
<div>
  <p>
    <strong>Important text</p>
  </strong>
</div>
```

1.8.4 Common attributes in all tags

Some attributes work in almost any tag:

Attribute	Description	Example
`id`	Unique element identifier	`id="section-1"`
`class`	CSS class for styling	`class="highlighted"`
`style`	Inline CSS styles	`style="color: red;"`
`title`	Text that appears on hover	`title="See more"`
`data-*`	Custom data	`data-number="42"`

1.8.5 DOCTYPE and HTML declaration

The **DOCTYPE** is not really a tag, but it's mandatory at the beginning of the document:

```
<!DOCTYPE html>
```

This tells the browser to use the HTML5 specification. In earlier HTML versions there were several variants, but in HTML5 there's only one.

Immediately after comes the root tag:

```
<!DOCTYPE html>
<html lang="en">
  ...
</html>
```

The `lang` attribute specifies the main language of the document (important for accessibility and SEO).

1.8.6 HTML validation

To verify that your HTML is correct, you can use the official W3C validator:

1. Go to https://validator.w3.org/
2. Enter your code or URL
3. The validator will show you errors and warnings

Keeping HTML valid is important for:

- Browser compatibility
- Accessibility
- SEO (search engine ranking)
- Code maintainability

THE HTML DOCUMENT: INTERNAL STRUCTURE

Every HTML page has a specific internal structure. Although you only see the content of the <body> in the browser, there's much more happening in the document's head (<head>).

2.1 Complete structure of an HTML document

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Title of my page</title>
    <link rel="icon" href="favicon.ico">
    <meta name="description" content="Brief description">
  </head>
  <body>
    <h1>Visible content here</h1>
  </body>
</html>
```

2.2 What is the <head>?

The <head> is the **document header**. It contains:

- **Metadata —** information about the page
- **Links —** to stylesheets, fonts, icons
- **Title —** what appears in the browser tab
- **Scripts —** JavaScript code
- **Instructions —** how the browser should interpret the content

The content of the <head> **doesn't appear on the page**, only the <body> does.

2.3 The `lang` attribute

Always specify the main language:

```html
<html lang="en">  <!-- English -->
<html lang="es">  <!-- Spanish -->
<html lang="fr">  <!-- French -->
```

This is important for:

- Search engines (SEO)
- Screen readers (accessibility)
- Browser spell checkers

2.4 Meta charset

Specifies the **character encoding** of the document:

```html
<meta charset="UTF-8">
```

UTF-8 is the modern standard and supports all languages and symbols. **Always use UTF-8**.

2.5 Meta viewport

Makes the page **responsive** (adapts to mobile screens):

```html
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Without this, your page would look bad on mobile devices.

2.6 Page title

```html
<title>My First Website</title>
```

The title appears in:

- The browser tab
- Search results
- Browser history

Important rule: each page should have a unique and descriptive title.

2.7 Favicon

The **favicon** is the small icon that appears in the browser tab:

```html
<link rel="icon" href="favicon.ico">
```

You can use:

- favicon.ico — traditional format
- favicon.png — modern format
- favicon.svg — vector (recommended)

2.8 Meta description

A brief description of your page that appears in search results:

```html
<meta name="description" content="Learn HTML from scratch. A practical guide for computer engineers.">
```

It should be 150-160 characters to appear complete in Google.

2.9 Meta tags for social media

Control how your page appears when shared on social media:

```html
<meta property="og:title" content="My First Website">
<meta property="og:description" content="Brief description">
<meta property="og:image" content="image.png">
<meta property="og:url" content="https://example.com">
```

These are **Open Graph** tags used by Facebook, Twitter, LinkedIn, etc.

2.10 Linking stylesheets

```html
<link rel="stylesheet" href="styles.css">
```

By convention, CSS links go in the <head>.

2.11 Scripts

```html
<script src="script.js"></script>
```

Although they can technically go in the <head> or at the end of the <body>, they're usually placed at the end of the <body> so the page loads faster.

The following chapters will dive deeper into each of these elements.

HTML SEMANTIC ELEMENTS

HTML semantics means using tags that correctly describe the meaning of their content. It's not just about how it looks in the browser, but about what that content *means*.

3.1 Why does semantics matter?

Using semantic HTML is important for:

1. **Search engines —** understand your content better for SEO
2. **Screen readers —** helps people with visual disabilities
3. **Maintainability —** your code is easier to understand and modify
4. **Accessibility —** you meet important web standards

3.2 Example: Semantic vs. Non-semantic

Incorrect (non-semantic):

```html
<div>
  <div><b>My Article</b></div>
  <div>Date: June 15</div>
  <div>The article content goes here...</div>
</div>
```

Correct (semantic):

```html
<article>
  <h1>My Article</h1>
  <time datetime="2025-06-15">June 15</time>
  <p>The article content goes here...</p>
</article>
```

3.3 Main semantic tags

3.3.1 General structure

- `<header>` — header of the page or section
- `<nav>` — navigation menu
- `<main>` — unique main content (only one per page)
- `<section>` — thematic section
- `<article>` — independent content (blog post, news, etc.)
- `<aside>` — sidebar or related content
- `<footer>` — page footer

3.3.2 Text and content

- `<h1>` to `<h6>` — headings (h1 is most important)
- `<p>` — paragraph
- `<strong>` — important text (not just bold)
- `<em>` — emphasis (not just italic)
- `<mark>` — highlighted text
- `<small>` — small text (notes, copyright)
- `<code>` — programming code
- `<pre>` — preformatted text

3.3.3 Lists

- `<ul>` — unordered list (bullets)
- `<ol>` — ordered list (numbers)
- `<li>` — list item
- `<dl>` — definition list
- `<dt>` — term to define
- `<dd>` — definition of term

3.3.4 Quotes and references

- `<blockquote>` — long quote
- `<q>` — short inline quote
- `<cite>` — reference to a work
- `<abbr>` — abbreviation

3.4 Structure of a semantic HTML page

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <title>My Blog</title>
</head>
<body>
  <header>
    <h1>My Website</h1>
    <nav>
      <a href="/">Home</a>
      <a href="/blog">Blog</a>
      <a href="/contact">Contact</a>
    </nav>
  </header>

  <main>
    <article>
      <h2>My First Article</h2>
      <p>Content...</p>
    </article>
  </main>

  <aside>
    <h3>Recent Articles</h3>
    <ul>
      <li><a href="#">Article 1</a></li>
      <li><a href="#">Article 2</a></li>
    </ul>
  </aside>

  <footer>
    <p>&copy; 2025 My Website</p>
  </footer>
</body>
</html>
```

3.5 Difference between `<strong>` and `<b>`

- `<strong>` — **important** text (semantic)
- `<b>` — **bold** text (presentation only)

Same with `<em>` vs `<i>`.

3.6 Accessibility and semantics

When you use semantic HTML:

- Screen readers understand the structure
- Search engines index your content better
- Your page is easier to maintain
- You comply with WCAG standards

Following chapters will show you how to use each of these tags in detail.

CHAPTER 4

LINKS AND HYPERLINKS

Links are the essence of the web. Without them, there would be no way to navigate from one page to another. In this section you'll learn how to create and manage links correctly.

4.1 The <a> tag

The `<a>` tag (anchor) creates a hyperlink. Its main attribute is `href` (Hypertext REFerence):

```
<a href="https://example.com">Click here</a>
```

The text between `<a>` and `</a>` is what the user sees and can click on.

4.2 URLs vs. Paths

There are two ways to specify destinations:

4.2.1 Absolute URLs

Complete web address:

```
<a href="https://www.google.com">Google</a>
<a href="http://example.com/page.html">External page</a>
```

4.2.2 Relative Paths

Paths within your own site:

```
<a href="page.html">Page in same directory</a>
<a href="folder/page.html">Page in a folder</a>
<a href="../page.html">Page in parent folder</a>
<a href="/">Home page</a>
```

4.3 Link attributes

Attribute	Use	Example
`href`	Link destination	`href="page.html"`
`target`	Where to open	`target="_blank"` (new tab)
`title`	Text on hover	`title="See more information"`
`rel`	Relation to destination	`rel="noopener noreferrer"`
`download`	Download file	`download="file.pdf"`

4.4 Open in new tab

```
<a href="https://example.com" target="_blank" rel="noopener noreferrer">
  Open in new tab
</a>
```

The `rel="noopener noreferrer"` attribute is important for security.

4.5 Internal links within the page

Use `id` to create links to sections:

```
<!-- Create a destination -->
<h2 id="section-2">Section 2</h2>

<!-- Create a link to that destination -->
<a href="#section-2">Go to Section 2</a>
```

4.6 Download files

```
<a href="document.pdf" download>Download PDF</a>
<a href="image.png" download="my-image.png">Download with different name</a>
```

4.7 Email and phone

```
<!-- Email link -->
<a href="mailto:user@example.com">Send email</a>

<!-- Phone link -->
<a href="tel:+34123456789">Call</a>

<!-- Email with parameters (add ?subject=topic to href) -->
<a href="mailto:user@example.com">Send inquiry</a>
```

4.8 Best practices

1. **Descriptive text —** use text that explains where the link goes, not "click here"
2. **Clear structure —** links should be easy to identify
3. **URL validation —** verify that links work
4. **Accessibility —** links must be navigable with keyboard

4.9 Difference between paths

Imagine this structure:

```
site/
  ├── index.html
  ├── page1.html
  └── folder/
      └── page2.html
```

From `folder/page2.html`:

- `../index.html` → go up one folder, access index.html
- `../page1.html` → go up one folder, access page1.html
- `./file.html` → in current folder
- `../../file.html` → go up two folders

The following chapters will teach you more about paths and how to organize them correctly.

TABLES IN HTML

Tables are fundamental elements for presenting tabular data in a structured way. Although they were historically used for page layouts (we use CSS for that today), tables remain essential for real data.

5.1 Basic table structure

```
<table>
  <tr>
    <td>Cell 1</td>
    <td>Cell 2</td>
  </tr>
  <tr>
    <td>Cell 3</td>
    <td>Cell 4</td>
  </tr>
</table>
```

5.2 Table tags

Tag	Meaning	Use
`<table>`	Defines the table	Main container
`<tr>`	Table Row	Each row of data
`<td>`	Table Data	Normal cells
`<th>`	Table Header	Header cells
`<thead>`	Table Head	Groups headers
`<tbody>`	Table Body	Groups data body
`<tfoot>`	Table Footer	Groups table footer
`<caption>`	Caption	Table title
`<colgroup>`	Column Group	Groups columns
`<col>`	Column	Defines a column

5.3 Table with complete structure

```
<table>
  <caption>Monthly Temperatures 2025</caption>
  <thead>
    <tr>
      <th>Month</th>
      <th>Min (°C)</th>
      <th>Max (°C)</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td>January</td>
      <td>5</td>
      <td>15</td>
    </tr>
    <tr>
      <td>February</td>
      <td>6</td>
      <td>16</td>
    </tr>
  </tbody>
  <tfoot>
    <tr>
      <td>Average</td>
      <td>5.5</td>
      <td>15.5</td>
    </tr>
  </tfoot>
</table>
```

5.4 Cell attributes

Attribute	Use	Example
`colspan`	Merge columns	`<td colspan="2">`
`rowspan`	Merge rows	`<td rowspan="3">`
`headers`	Associate to headers	`<td headers="col1 col2">`
`scope`	Header scope	`<th scope="col">`

5.5 Merged cells

```
<table>
  <tr>
    <th>Name</th>
    <th colspan="2">Grades</th>
  </tr>
  <tr>
    <td>John</td>
```

(continues on next page)

(continued from previous page)

```
    <td>8.5</td>
    <td>9.0</td>
  </tr>
</table>
```

5.6 Accessibility in tables

To make tables accessible for screen readers:

```
<table>
  <thead>
    <tr>
      <th scope="col" id="month">Month</th>
      <th scope="col" id="temp">Temperature</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td headers="month">January</td>
      <td headers="temp">15°C</td>
    </tr>
  </tbody>
</table>
```

5.7 Complex tables

Complex tables with multiple levels of headers can be hard to make accessible. In those cases, consider:

1. **Simplify the table** if possible
2. **Split it into simpler tables**
3. **Use accessibility attributes** like `scope` and `headers`
4. **Provide alternatives** (descriptive text, list, etc.)

5.8 When to use tables?

Use them for:

- Real tabular data
- Comparisons
- Statistics
- Schedules

DON'T use them for:

- Page layouts (use CSS Grid or Flexbox)
- Non-tabular content

- Visual effects

In upcoming chapters you'll learn to style tables with CSS to make them more visually appealing.

YOUR FIRST HTML PROJECT

Now that you've learned the fundamentals, it's time to create your first real web page. We'll create a simple but complete website that combines everything we've covered.

6.1 Project Objective

You'll create a **personal website** that includes:

- Correct HTML5 structure
- Multiple semantic sections
- Functional navigation
- Properly formatted content
- Internal and external links

6.2 Project Structure

```
my-website/
  ├── index.html
  ├── about-me.html
  ├── projects.html
  ├── contact.html
  └── css/
      └── styles.css
```

6.3 The index.html file

```html
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Personal Page - John Developer</title>
  <meta name="description" content="Welcome to my personal page. I'm a web developer passionate about technology.">
  <link rel="stylesheet" href="css/styles.css">
```

(continues on next page)

(continued from previous page)

```
</head>
<body>
  <!-- Header and navigation -->
  <header>
    <h1>John Developer</h1>
    <nav>
      <a href="index.html">Home</a>
      <a href="about-me.html">About Me</a>
      <a href="projects.html">Projects</a>
      <a href="contact.html">Contact</a>
    </nav>
  </header>

  <!-- Main content -->
  <main>
    <section id="introduction">
      <h2>Welcome</h2>
      <p>Hello, I'm <strong>John</strong>, a Computer Engineering student passionate␣
↪about web development.</p>
      <p>On this site you'll find information about me and my projects.</p>
    </section>

    <section id="highlights">
      <h2>Highlights</h2>
      <ul>
        <li>Programming in HTML, CSS and JavaScript</li>
        <li>Web application development</li>
        <li>Responsive design</li>
      </ul>
    </section>

    <section id="knowledge">
      <h2>My Knowledge</h2>
      <table>
        <thead>
          <tr>
            <th>Language</th>
            <th>Level</th>
          </tr>
        </thead>
        <tbody>
          <tr>
            <td>HTML</td>
            <td>Intermediate</td>
          </tr>
          <tr>
            <td>CSS</td>
            <td>Basic</td>
          </tr>
          <tr>
            <td>JavaScript</td>
            <td>Basic</td>
          </tr>
        </tbody>
      </table>
    </section>
  </main>
```

(continues on next page)

(continued from previous page)

```
  <!-- Sidebar (optional) -->
  <aside>
    <h3>Useful Links</h3>
    <ul>
      <li><a href="https://github.com" target="_blank" rel="noopener noreferrer">My␣
↪GitHub</a></li>
      <li><a href="https://linkedin.com" target="_blank" rel="noopener noreferrer">My␣
↪LinkedIn</a></li>
      <li><a href="mailto:john@example.com">Email me</a></li>
    </ul>
  </aside>

  <!-- Footer -->
  <footer>
    <p>&copy; 2025 John Developer. All rights reserved.</p>
    <p><small>Last updated: June 2025</small></p>
  </footer>
</body>
</html>
```

6.4 Other pages

6.4.1 about-me.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>About Me - John Developer</title>
  <link rel="stylesheet" href="css/styles.css">
</head>
<body>
  <header>
    <h1>John Developer</h1>
    <nav>
      <a href="index.html">Home</a>
      <a href="about-me.html">About Me</a>
      <a href="projects.html">Projects</a>
      <a href="contact.html">Contact</a>
    </nav>
  </header>

  <main>
    <article>
      <h2>About Me</h2>
      <p>I'm a Computer Engineering student at the University...</p>
      <h3>My Journey</h3>
      <p>I started learning programming 3 years ago...</p>
    </article>
  </main>
```

(continues on next page)

(continued from previous page)

```
  <footer>
    <p>&copy; 2025 John Developer.</p>
  </footer>
</body>
</html>
```

6.5 Steps to create your project

1. **Create a folder** called `my-website`
2. **Copy the code** from `index.html` into a new file
3. **Open the page** in your browser (File → Open)
4. **Create the other pages** (`about-me.html`, `projects.html`, `contact.html`)
5. **Verify that links** work between pages
6. **Validate your HTML** at validator.w3.org

6.6 Next steps

Once you have the basic HTML working:

1. **Learn CSS** to style your page
2. **Add JavaScript** for interactivity
3. **Publish your site** on GitHub Pages or a web server
4. **Keep improving** your project

6.7 Review Checklist

- [ ] All files are in UTF-8
- [ ] You have a correct DOCTYPE
- [ ] Navigation works between pages
- [ ] External links open in new tab (with `target="_blank"`)
- [ ] Your HTML validates without errors
- [ ] Relative paths work correctly
- [ ] You used semantic tags (`<header>`, `<main>`, `<footer>`, etc.)

Congratulations! You've created your first website. Now it's time to style it and make it more interesting.